Tercer examen parcial

5. Piedra, papel o tijera

Jonathan Serna

Programación Asistida por Inteligencia Artificial

6 de noviembre de 2026

1. Objetivo, diseño y ejecución

Implementar el ejercicio 5: el jugador elige piedra, papel o tijera; la computadora elige al azar y se resuelve una ronda. La interfaz usa Streamlit y muestra las victorias del jugador, las de la computadora y los empates.

Se eligieron dos módulos: **game.py** para las reglas y **app.py** para la pantalla y el estado.

Contrato y reglas

determine_winner(player_move: str, computer_move: str) -> str. Acepta exactamente **piedra**, **papel** o **tijera**. Devuelve **player**, **computer** o **tie**. Si cualquiera de las jugadas es inválida, lanza **ValueError**.

Piedra vence a tijera, papel vence a piedra y tijera vence a papel. Dos jugadas iguales empatan. La elección aleatoria y el aumento del marcador ocurren una vez por clic en **Jugar ronda**.

Instalación y uso

Requisitos: Python 3.14 y uv. Dentro de **28vo Tercer Parcial**, ejecutar:

```
uv sync --locked
uv run streamlit run app.py
uv run python -m pytest -v
```

Abrir la dirección local indicada por Streamlit. Seleccionar una jugada y pulsar el botón. El marcador pertenece a la sesión; recargar o cerrar la página puede reiniciarlo. No se añadió base de datos ni funciones extras.

Código funcional: abrir proyecto en GitHub.

2. Diagrama de flujo en Mermaid

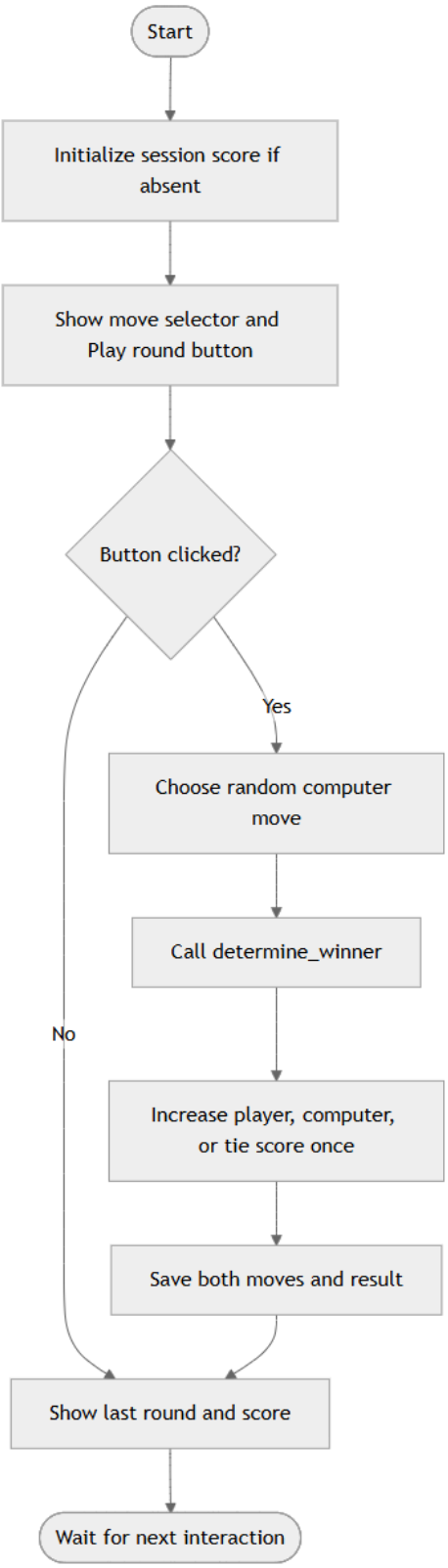
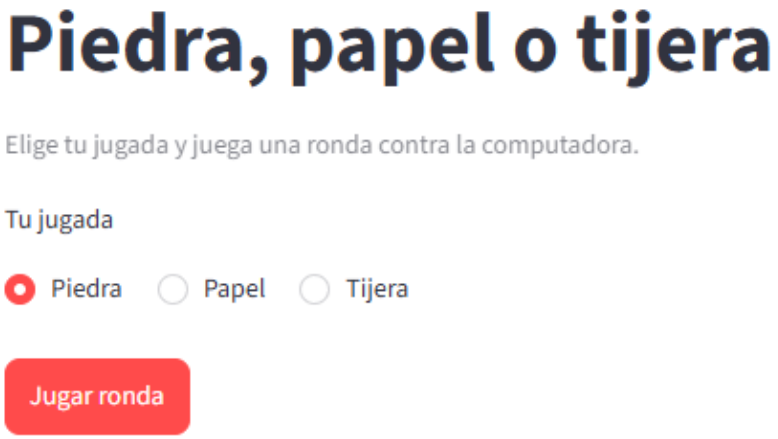
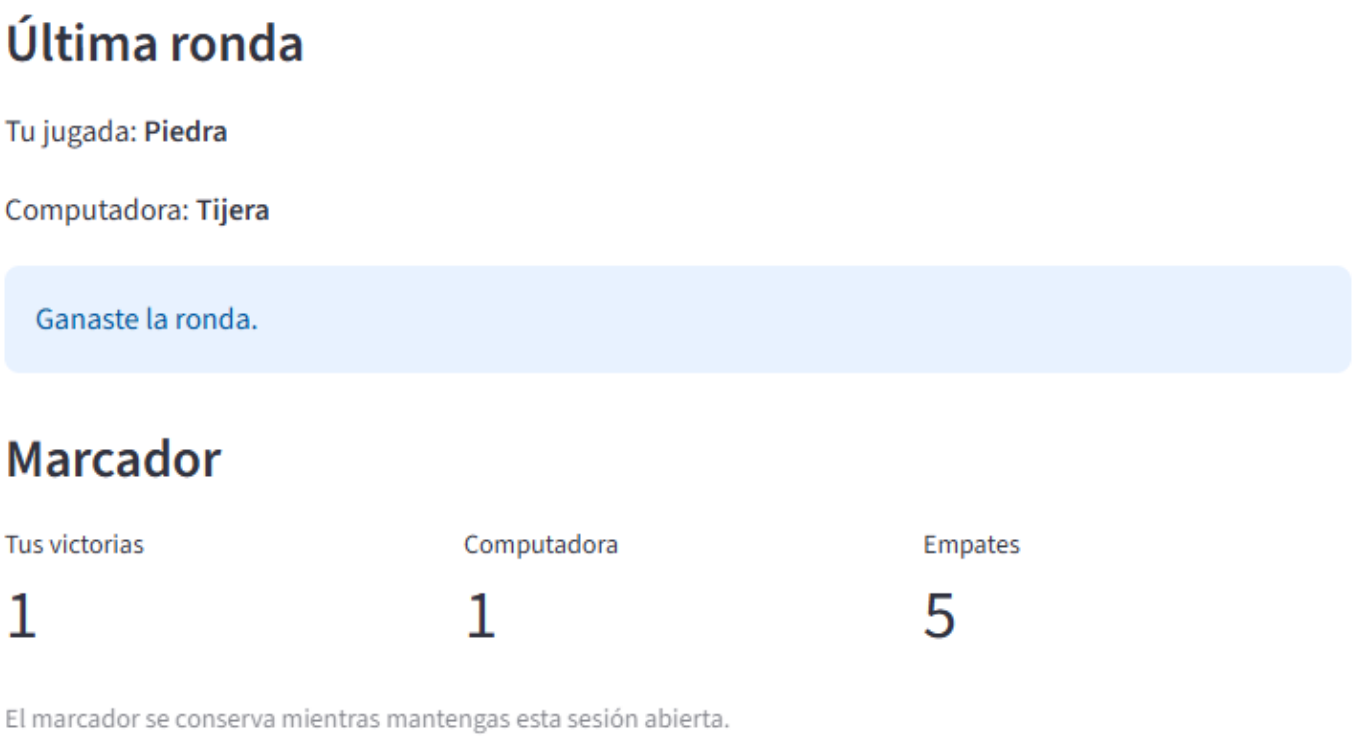


Diagrama renderizado desde docs/flowchart.mmd. La selección por sí sola no juega una ronda; el botón actualiza un único contador.

3. Interfaz y resultado de una ronda

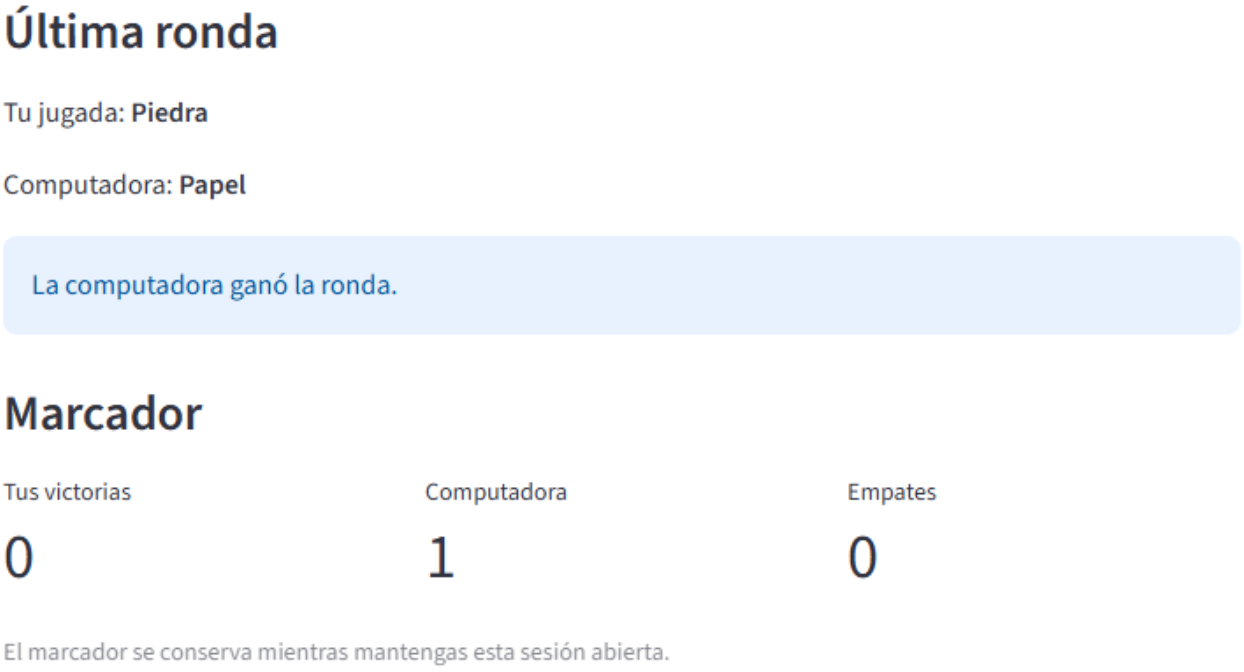


Selección de piedra, papel o tijera y botón para jugar.

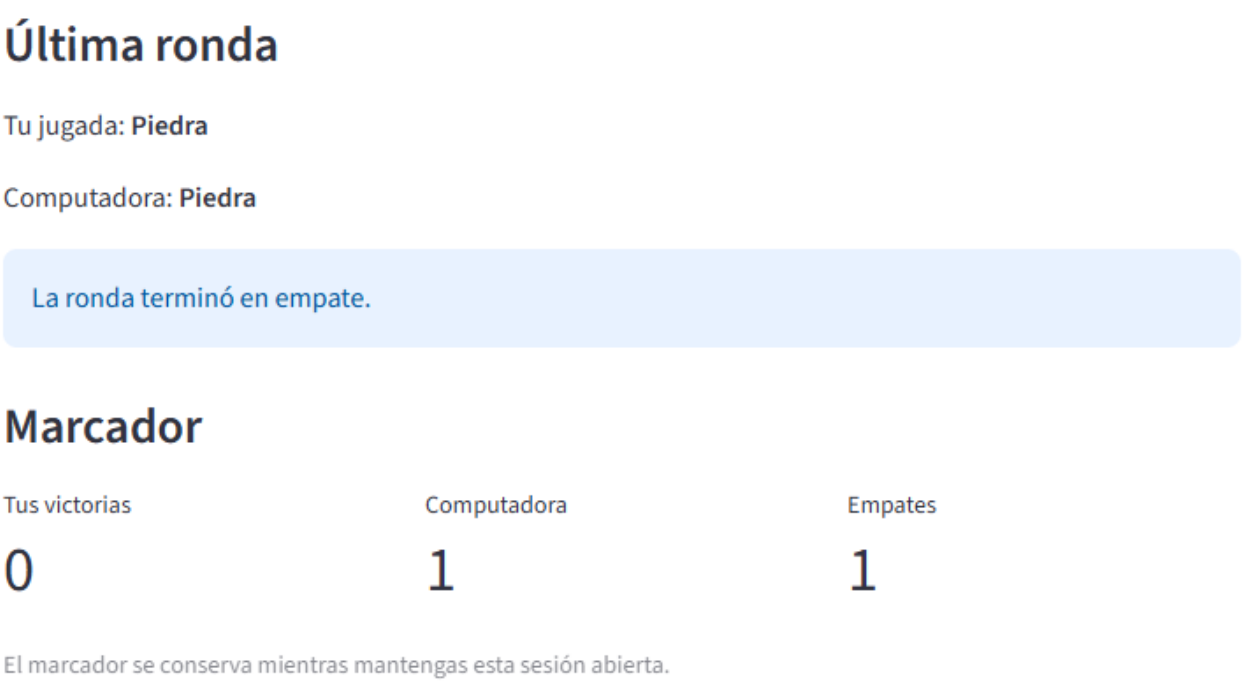


Victoria del jugador: piedra contra tijera. Se muestran ambas jugadas, el resultado y los tres contadores.

4. Derrota, empate y marcador



La computadora gana: papel contra piedra del jugador.



Empate: ambas jugadas son piedra. Las capturas proceden de rondas reales; no se forzó la elección aleatoria.

5. Pruebas escritas antes de implementar

Las expectativas salen de las reglas: nueve pares válidos y diez entradas inválidas. La función inicial solo lanzaba NotImplementedError. Las mismas pruebas se ejecutaron en rojo y en verde, sin cambiar sus expectativas.

tests/test_game.py - mismo archivo en ambas fases

```
# -*- coding: utf-8 -*-
# @Author: Jonathan Serna
# @Date:   2026-11-06 01:21:26
# @Last Modified by:   Jonathan Serna
# @Last Modified time: 2026-11-06 01:21:26
"""Check the winner contract independently of the implementation."""

import pytest

from game import determine_winner


@pytest.mark.parametrize("player_move, computer_move, expected", [
    ("piedra", "piedra", "tie"),
    ("piedra", "papel", "computer"),
    ("piedra", "tijera", "player"),
    ("papel", "piedra", "player"),
    ("papel", "papel", "tie"),
    ("papel", "tijera", "computer"),
    ("tijera", "piedra", "computer"),
    ("tijera", "papel", "player"),
    ("tijera", "tijera", "tie"),
])
def test_determine_winner(player_move, computer_move, expected):
    """Check every valid pair against the rules."""
    assert determine_winner(player_move, computer_move) == expected


@pytest.mark.parametrize("invalid_move", ["", "lagarto", "Piedra", None, []])
@pytest.mark.parametrize("position", ["player", "computer"])
def test_invalid_move(invalid_move, position):
    """Reject an invalid value in either position with ValueError."""
    player_move = invalid_move if position == "player" else "piedra"
    computer_move = invalid_move if position == "computer" else "piedra"
    with pytest.raises(ValueError):
        determine_winner(player_move, computer_move)
```

Código completo de tests/test_game.py usado en las dos fases.

6. TDD: fase roja

Comando: **python -m pytest tests/test_game.py -v --tb=short --color=no**. Resultado real: **19 failed**, por NotImplementedError; código de salida 1.

Salida de pytest guardada durante la fase rojo

```
============================= test session starts =============================
platform win32 -- Python 3.14.5, pytest-9.2.0, pluggy-1.7.0 -- E:\Universidad\Programacion-Asistida\28vo
Tercer Parcial\.venv\Scripts\python.exe
cachedir: .pytest_cache
rootdir: E:\Universidad\Programacion-Asistida\28vo Tercer Parcial
configfile: pyproject.toml
plugins: anyio-4.16.0
collecting ... collected 19 items

tests/test_game.py::test_determine_winner[piedra-piedra-tie] FAILED      [  5%]
tests/test_game.py::test_determine_winner[piedra-papel-computer] FAILED  [ 10%]
tests/test_game.py::test_determine_winner[piedra-tijera-player] FAILED   [ 15%]
tests/test_game.py::test_determine_winner[papel-piedra-player] FAILED    [ 21%]
tests/test_game.py::test_determine_winner[papel-papel-tie] FAILED        [ 26%]
tests/test_game.py::test_determine_winner[papel-tijera-computer] FAILED  [ 31%]
tests/test_game.py::test_determine_winner[tijera-piedra-computer] FAILED [ 36%]
tests/test_game.py::test_determine_winner[tijera-papel-player] FAILED    [ 42%]
tests/test_game.py::test_determine_winner[tijera-tijera-tie] FAILED      [ 47%]
tests/test_game.py::test_invalid_move[player-] FAILED                    [ 52%]
tests/test_game.py::test_invalid_move[player-lagarto] FAILED             [ 57%]
tests/test_game.py::test_invalid_move[player-Piedra] FAILED              [ 63%]
tests/test_game.py::test_invalid_move[player-None] FAILED                [ 68%]
tests/test_game.py::test_invalid_move[player-invalid_move4] FAILED       [ 73%]
tests/test_game.py::test_invalid_move[computer-] FAILED                  [ 78%]
tests/test_game.py::test_invalid_move[computer-lagarto] FAILED           [ 84%]
tests/test_game.py::test_invalid_move[computer-Piedra] FAILED            [ 89%]
tests/test_game.py::test_invalid_move[computer-None] FAILED              [ 94%]
tests/test_game.py::test_invalid_move[computer-invalid_move4] FAILED     [100%]

================================== FAILURES ===================================
___________________ test_determine_winner[piedra-piedra-tie] ___________________
tests\test_game.py:26: in test_determine_winner
    assert determine_winner(player_move, computer_move) == expected
           ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
game.py:11: in determine_winner
    raise NotImplementedError
E   NotImplementedError
_________________ test_determine_winner[piedra-papel-computer] __________________

[Extracto; salida completa en tdd-red.txt]
============================= 19 failed in 0.93s ==============================
```

Vista de la salida real guardada. Se muestra el resumen y el primer fallo; tdd-red.txt conserva la salida completa.

7. TDD: fase verde y verificación final

Se implementaron la validación, el empate y las tres parejas ganadoras. El mismo comando produjo **19 passed**, con código de salida 0. No se modificó tests/test_game.py.

Salida de pytest guardada durante la fase verde

```
============================= test session starts =============================
platform win32 -- Python 3.14.5, pytest-9.2.0, pluggy-1.7.0 -- E:\Universidad\Programacion-Asistida\28vo
Tercer Parcial\.venv\Scripts\python.exe
cachedir: .pytest_cache
rootdir: E:\Universidad\Programacion-Asistida\28vo Tercer Parcial
configfile: pyproject.toml
plugins: anyio-4.16.0
collecting ... collected 19 items

tests/test_game.py::test_determine_winner[piedra-piedra-tie] PASSED      [  5%]
tests/test_game.py::test_determine_winner[piedra-papel-computer] PASSED  [ 10%]
tests/test_game.py::test_determine_winner[piedra-tijera-player] PASSED   [ 15%]
tests/test_game.py::test_determine_winner[papel-piedra-player] PASSED    [ 21%]
tests/test_game.py::test_determine_winner[papel-papel-tie] PASSED        [ 26%]
tests/test_game.py::test_determine_winner[papel-tijera-computer] PASSED  [ 31%]
tests/test_game.py::test_determine_winner[tijera-piedra-computer] PASSED [ 36%]
tests/test_game.py::test_determine_winner[tijera-papel-player] PASSED    [ 42%]
tests/test_game.py::test_determine_winner[tijera-tijera-tie] PASSED      [ 47%]
tests/test_game.py::test_invalid_move[player-] PASSED                    [ 52%]
tests/test_game.py::test_invalid_move[player-lagarto] PASSED             [ 57%]
tests/test_game.py::test_invalid_move[player-Piedra] PASSED              [ 63%]
tests/test_game.py::test_invalid_move[player-None] PASSED                [ 68%]
tests/test_game.py::test_invalid_move[player-invalid_move4] PASSED       [ 73%]
tests/test_game.py::test_invalid_move[computer-] PASSED                  [ 78%]
tests/test_game.py::test_invalid_move[computer-lagarto] PASSED           [ 84%]
tests/test_game.py::test_invalid_move[computer-Piedra] PASSED            [ 89%]
tests/test_game.py::test_invalid_move[computer-None] PASSED              [ 94%]
tests/test_game.py::test_invalid_move[computer-invalid_move4] PASSED     [100%]

============================== 19 passed in 0.12s ==============================
```

Salida real de la fase verde. Las capturas comparativas completas con código y salida están en docs/evidence/tdd-red.png y tdd-green.png.

Comprobación final: **22 pruebas aprobadas**. AppTest verificó los tres resultados, dos clics sucesivos, la selección sin sumar puntos y sesiones independientes. También se revisaron sintaxis, docstrings y estilo básico.

SHA-256 del archivo de pruebas, idéntico antes y después de implementar:
30CE339EEEC91A70A7B107BB43920856FCA786A1B01F59F90647EBC0BA9F70D9.

Las imágenes de TDD muestran el archivo y la salida guardada de ejecuciones reales. Son vistas de esa evidencia, no fotografías de una terminal nativa. Los archivos de texto permiten revisar todos los fallos y resultados.

8. Diálogo ilustrativo

Simulación didáctica basada en el proyecto. Los intercambios siguientes son ejemplos de solicitudes, preguntas y ajustes. No son una transcripción de una sesión real. Los nombres de archivos, las reglas y los resultados de pruebas mencionados corresponden al código y las evidencias de la entrega.

1. Contexto y delimitación del ejercicio

Estudiante: Necesito resolver el ejercicio 5 del tercer parcial: piedra, papel o tijera con Python y Streamlit. El jugador elige una jugada, la computadora elige otra al azar y se muestra quién ganó. También necesito contar mis victorias, las de la computadora y los empates. Primero revisemos las reglas y el diseño; todavía no escribas código.

Asistente: El alcance puede limitarse a una sola pantalla. Antes del diseño conviene precisar dos cosas: ¿el marcador debe sobrevivir al cierre de la página? ¿La función aceptará variantes como "Piedra" con mayúscula, o únicamente los tres valores exactos que maneje la interfaz?

Estudiante: Que el marcador dure durante la sesión; no quiero una base de datos. La pantalla estará en español. Usemos exactamente "piedra", "papel" y "tijera" como valores internos. Si la función recibe otra cosa, debe rechazarla. No agregues cuentas, historial de partidas ni exportaciones.

Asistente: Entonces usaremos controles nativos de Streamlit y estado de sesión. Las etiquetas podrán empezar con mayúscula, pero los valores enviados a la función seguirán en minúscula. El contrato distinguirá tres resultados: "player", "computer" y "tie". Las entradas inválidas producirán ValueError; no se corregirán silenciosamente.

Estudiante: Confirma las reglas para que después las pruebas no terminen aceptando un error del programa. Piedra gana a tijera, papel gana a piedra, tijera gana a papel y dos jugadas iguales empatan. La computadora debe elegir otra vez al pulsar el botón, aunque repita la jugada anterior.

Asistente: De acuerdo. Una elección aleatoria puede repetirse; eso no significa que el botón haya fallado. Cada clic representa una ronda y suma exactamente un punto a uno de los tres contadores. Cambiar la selección no representa una ronda y no debe modificar el marcador.

2. Diseño antes del código y diagrama Mermaid

Estudiante: Propón ahora la organización de archivos y lo que se verá en la pantalla. Mantén la lógica del ganador separada de Streamlit para poder probarla directamente. Incluye el diagrama de flujo en Mermaid antes de implementar.

Asistente: Propongo game.py con los movimientos válidos y la función determine_winner, y app.py con la interfaz, la elección aleatoria y el marcador. Las pruebas irán en tests/. La pantalla tendrá título, selector horizontal de jugada, botón "Jugar ronda", última ronda y tres métricas: "Tus victorias", "Computadora" y "Empates".

Estudiante: En el diagrama quiero ver qué sucede cuando solamente cambio el selector. También debe quedar claro que el marcador se prepara una vez por sesión y que no se vuelve a poner en cero después de cada clic.

Asistente: El flujo Mermaid de docs/flowchart.mmd empieza comprobando el estado de sesión. Después muestra los controles y pregunta si se pulsó el botón. La rama "No" muestra el resultado anterior y el marcador. La rama "Sí" elige la jugada de la computadora, llama a la función, incrementa un contador, guarda ambas jugadas y muestra el resultado. El render está en docs/flowchart.svg y docs/flowchart.png.

Estudiante: Con esa estructura es suficiente. Puedes tomar el diseño como aprobado. Antes de construir la pantalla, preparemos el contrato y las pruebas de la función. No quiero que una corrección de interfaz obligue a cambiar las reglas del juego.

Asistente: La separación permite hacerlo: game.py no necesita importar Streamlit ni elegir al azar. Recibe dos jugadas y decide el resultado. app.py se encarga de obtenerlas y actualizar el estado. El diagrama describe ese recorrido y las pruebas fijarán el comportamiento del contrato.

3. Especificación de las pruebas

Estudiante: El contrato será exactamente determine_winner(player_move: str, computer_move: str) -> str. Escribe primero las pruebas con pytest a partir de estas reglas, sin deducir los valores esperados de una implementación. Agrupa los casos semejantes con @pytest.mark.parametrize y no cambies las expectativas para conseguir que una prueba pase.

Asistente: Para las nueve combinaciones válidas, los tres empates son piedra/piedra, papel/papel y tijera/tijera. Las victorias del jugador son piedra/tijera, papel/piedra y tijera/papel. Las derrotas son piedra/papel, papel/tijera y tijera/piedra. El primer valor siempre representa al jugador; los resultados esperados serán "tie", "player" y "computer", respectivamente.

Estudiante: Además de una palabra inválida, prueba una cadena vacía y una jugada con mayúscula. Comprueba los dos parámetros: no bastaría con rechazar una entrada incorrecta del jugador si la de la computadora pasa sin validarse. ¿Podemos revisar también un valor que no sea texto?

Asistente: Sí. tests/test_game.py usa cinco valores inválidos: una cadena vacía, "lagarto", "Piedra", None y una lista vacía. Cada valor se prueba en la posición del jugador y de la computadora, dejando válida la otra jugada. Son diez casos con pytest.raises(ValueError), además de los nueve pares válidos: diecinueve casos en total.

Estudiante: Bien. Para la fase roja, la función debe existir con su firma y docstring, pero su cuerpo debe limitarse a raise NotImplementedError. Ejecuta las pruebas antes de implementar y guarda tanto el código de prueba como la salida. No uses skip, xfail ni excepciones que oculten los fallos.

Asistente: El punto de partida queda separado en la evidencia docs/evidence/game-red.py.txt; el archivo de pruebas usado se conserva en test-game.py.txt. El comando de esta fase es python -m pytest tests/test_game.py -v --tb=short --color=no. Así puede revisarse qué se ejecutó y qué estaba pendiente de implementar.

4. Fase roja: revisar el fallo esperado

Estudiante: Al revisar la salida veo "19 failed". Antes de cambiar algo, explica el motivo concreto del fallo. Quiero distinguir un error esperado del contrato vacío de un problema de importación o de una prueba mal escrita.

Asistente: En docs/evidence/tdd-red.txt se recogen diecinueve pruebas y los fallos terminan en NotImplementedError dentro de game.py. La función se importa y las pruebas se ejecutan; lo que falta es el comportamiento. La salida tiene código 1. No es necesario modificar las expectativas para explicar ese resultado.

Estudiante: Conserva esa ejecución como la evidencia roja. Quiero una captura legible del archivo de pruebas y otra de la salida, además de la vista comparativa. Después de implementar deben ser las mismas pruebas, no una versión más fácil.

Asistente: Las evidencias incluyen tdd-red.png, tdd-tests-detail.png y tdd-red-output.png. La salida completa permanece en el archivo de texto. El SHA-256 del archivo de pruebas se guarda en test-hash-red.txt para compararlo con el registrado después de la implementación.

5. Fase verde y ajuste de legibilidad

Estudiante: Implementa únicamente determine_winner para hacer pasar esas pruebas. Primero valida las jugadas; luego resuelve empate, victoria y derrota. No edites tests/test_game.py, no cambies sus resultados esperados y no agregues funciones que todavía no necesitamos.

Asistente: La solución de game.py comprueba que ambas jugadas estén en MOVES, que contiene los tres valores permitidos. Si alguna no está, lanza ValueError. Si son iguales, devuelve "tie". En los otros casos compara el par con las tres parejas ganadoras; devuelve "player" si coincide y "computer" si no coincide.

Estudiante: ¿Qué pasó al repetir el mismo comando? Comprueba también que no se haya perdido la validación del segundo argumento y que ninguna prueba haya sido omitida.

Asistente: La salida guardada en docs/evidence/tdd-green.txt contiene "19 passed" y código de salida 0. Incluye los nueve pares válidos y las diez entradas inválidas en ambas posiciones. test-hash-green.txt coincide con test-hash-red.txt: el archivo de pruebas no cambió entre esas dos fases.

Estudiante: Ahora revisa la claridad. La decisión final entre "player" y "computer" resulta más fácil de seguir con un if y dos retornos que con una expresión demasiado larga. Conserva la firma, la validación y el comportamiento. Este ajuste debe ser una refactorización, no otra regla.

Asistente: La versión final usa if (player_move, computer_move) in winning_pairs para devolver "player" y, después, devuelve "computer". Mantiene una sola responsabilidad y su docstring. El cambio de presentación no modifica las parejas ganadoras ni el tipo de error para entradas inválidas; la comprobación final vuelve a ejecutar las pruebas.

6. Construcción de la interfaz y revisión del estado

Estudiante: Pasemos a Streamlit. Quiero elegir la jugada con un selector y jugar solamente al pulsar "Jugar ronda". Deben verse mi elección, la de la computadora y el resultado. No quiero que al cambiar de opción se genere una nueva ronda automáticamente.

Asistente: app.py usa st.radio con los valores de MOVES y un botón. La llamada a random.choice(MOVES) queda dentro de la condición del botón, junto con la llamada a determine_winner. El resultado se usa como clave del diccionario de contadores para incrementar solamente el que corresponde.

Estudiante: Revisión de esta iteración: meta original, jugar una ronda y mostrar el marcador; refinamiento anterior, lógica ya probada; estado actual, controles y actualización por integrar; nuevo feedback, conservar el último resultado al cambiar de selección; restricciones, no tocar las reglas ni añadir persistencia. ¿Cómo lo resolvemos sin complicar la aplicación?

Asistente: st.session_state contiene scores y last_round. Los contadores solo se inicializan si todavía no existen. last_round guarda una tupla con la jugada del jugador, la de la computadora y el ganador. El renderizado consulta esa tupla aunque el selector provoque otra ejecución del script, sin volver a sumar puntos.

Estudiante: Antes de dar esa parte por terminada, revisa el orden de la pantalla. El marcador debe mostrar el punto nuevo en el mismo clic, no hasta la siguiente interacción. Al abrir una sesión tampoco debe aparecer una última ronda inventada. Aclara qué ocurre si se recarga la página.

Asistente: Los contadores se muestran después de procesar el botón, por eso reflejan la actualización inmediatamente. La sección "Última ronda" solo aparece cuando last_round es distinta de None. El texto inferior indica que el marcador se conserva mientras siga abierta esa sesión; no se promete conservarlo al recargar, cerrar la página o reiniciar el servidor.

7. Pruebas de integración de la pantalla

Estudiante: Las pruebas del ganador no comprueban el marcador. Añade una verificación pequeña de la interfaz para los tres resultados. Debe detectar si el marcador se reinicia, si suma dos veces por clic o si cambiar la selección modifica el resultado anterior.

Asistente: tests/test_app.py usa AppTest, incluido en Streamlit, y tres casos parametrizados. En cada caso monkeypatch controla random.choice durante la prueba: tijera permite comprobar la victoria de piedra, papel la derrota y piedra el empate. Esto vuelve repetible la prueba sin modificar la elección aleatoria de la aplicación de uso normal.

Estudiante: ¿Qué verificas después del primer clic? Me interesa comprobar lo que ve el usuario, no solamente que haya una variable con el valor uno. Incluye el texto del resultado, las jugadas y el contador correcto.

Asistente: Cada caso parte de tres contadores en cero y comprueba que no haya excepciones. Después del clic revisa el mensaje de resultado, la tupla de la última ronda, la jugada mostrada de la computadora y que el contador correspondiente sea uno. La suma de los tres contadores también debe ser uno. En la ronda inicial el jugador tiene seleccionada piedra.

Estudiante: Completa la revisión con otra interacción. Cambia el selector a papel sin jugar, comprueba que nada se sume y luego juega otra vez con piedra. También abre otra sesión para comprobar que no se mezclen los marcadores de dos personas.

Asistente: La prueba conserva el marcador y last_round al cambiar el selector; con el segundo clic comprueba dos puntos en el contador esperado y dos rondas en total. Una instancia nueva de AppTest empieza con cero puntos y sin última ronda. El conjunto completo registra "22 passed": diecinueve casos del contrato y tres de la interfaz.

8. Capturas y revisión de la evidencia

Estudiante: Para el documento necesito capturas de las funcionalidades: los controles, una victoria, una derrota, un empate y el marcador. Conserva también el diagrama y las capturas roja y verde con el código de pruebas y su salida. No uses una imagen inventada para mostrar el juego funcionando.

Asistente: Las capturas de la aplicación están en docs/evidence/: ui-controls.png, ui-player-detail.png, ui-computer-detail.png y ui-tie-detail.png, además de las vistas completas. Proceden de rondas reales del navegador. La documentación de TDD incluye las vistas de código y las salidas guardadas de pytest; el texto completo permite revisar lo que no cabe en una captura.

Estudiante: Distingue la comprobación automática de la manual. En las pruebas se controla el azar para repetir un caso; en las capturas de uso normal quiero que la computadora siga eligiendo al azar. Revisa que las imágenes se lean bien al incorporarlas al PDF.

Asistente: El control del azar se limita a AppTest. Las capturas del navegador muestran resultados obtenidos jugando, sin forzarlos. El PDF separa el diagrama, las funcionalidades, el código de pruebas, la fase roja y la fase verde. Los acercamientos permiten leer las jugadas, el marcador y la salida sin cambiar el contenido capturado.

9. Preparación y revisión de la entrega

Estudiante: Deja instrucciones para que el profesor pueda ejecutar el código desde el repositorio. El documento debe contener el enlace al código, el diagrama, el material de conversación y las evidencias. Revisa esos apartados contra la rúbrica antes de prepararlo para mi aprobación.

Asistente: El README indica Python 3.14 y los comandos uv sync --locked, uv run streamlit run app.py y uv run python -m pytest -v. pyproject.toml declara Streamlit y pytest; uv.lock fija las versiones. El enlace al proyecto está en el primer apartado del PDF. El diagrama tiene fuente Mermaid y render, y las funciones y TDD tienen sus capturas.

Estudiante: Conserva el proyecto mínimo que acordamos. No agregues un botón de reinicio, otra pantalla o una base de datos solo para hacer más larga la entrega. Quiero revisar el PDF antes de que se haga el commit del documento final.

Asistente: El juego conserva dos módulos y controles nativos. La revisión de entrega se apoya en el código, las diecinueve pruebas del contrato, las tres de interfaz y las evidencias guardadas. El PDF reúne el diagrama, los apartados de desarrollo y las capturas; se presenta para revisión y su commit queda pendiente de aprobación.

9. Referencias

Tercer examen parcial - Programación asistida por IA (editable), ejercicio 5, página 5. Dr. Miguel Jara Maldonado.

Módulo 6. Verificación, pruebas y depuración: pruebas basadas en especificación y ciclo rojo-verde-refactor. Se consultaron especialmente las páginas 14, 16 y 28-43.

Los PDF de ejercicios anteriores del repositorio se usaron como referencia de portada, apartados y evidencia.

Streamlit: Session State; AppTest; Mermaid: renderizado.

Este PDF queda pendiente de revisión del estudiante antes de su segundo commit. No se envió por correo.